

STRUCTTECH OS

Design & App Suite Blueprint

One modular, multi-tenant platform that runs StructTech's own business and is licensed, in configurable pieces, to StructTech's clients.

Status: IA locked from wireframes (turns 1a–1e, 2a–2i, 3a). Hi-fi system pass complete.

Locked Visual Source of Truth: StructTech OS - HiFi (standalone).html + wireframes

Current Phase: Week 1–3 COMPLETE — Foundation + Pipeline + BMR Estimating + Coordination + Field + CRM Depth (Lead Control Center)

Build Directive: CLAUDE.md (non-negotiable rules, patterns, multi-tenancy model)

Master Scope: SCOPE.md v1 (locked 7/8/26)

Backlog: BACKLOG.md (durable queue, last updated 7/13/26)

Design Tokens (exact from CLAUDE.md)

--bg: oklch(98% 0.004 90) --surface: oklch(99.5% 0.002 90) --surface2: oklch(95.3% 0.004 90)
--text: oklch(22% 0.006 90) --muted: oklch(48% 0.006 90) --border: oklch(88% 0.006 90)
--accent: oklch(0.55 0.16 250) --accent-strong: oklch(0.42 0.16 250) --accent-soft: oklch(0.93 0.03 250)
--warn: oklch(0.55 0.15 45) --warn-soft: oklch(0.94 0.04 45)

Fonts: IBM Plex Sans (UI) · IBM Plex Mono (money, measurements, IDs) · IBM Plex Serif (client-portal headings only)

"One build, licensed per tenant. Differences are config (entitlements + flags), not code."

"Business logic lives in server actions / RPCs, never buried in components — this action layer becomes the AI assistant's tool surface later."

EXECUTIVE SUMMARY & VISION

What StructTech OS is: One modular, multi-tenant platform that runs StructTech's own business *and* is licensed, in configurable pieces, to StructTech's clients. **Not two products — one build**, where each tenant is granted only the modules it pays for.

StructTech is simply the **first and most-entitled tenant** of its own OS. A contractor client (e.g. Brothers Metal Roofing) is a tenant with a subset of modules. Onboarding a new client is a **config action — inserting rows — not an engineering project**.

The business goal it serves: Productized delivery that carries a client from **scan** → **sold** → **delivered** → **proven** → **testimonial**, with end-to-end clarity, driving toward **\$20K MRR** (3 consecutive months + 3 testimonials from transitioned engagements). MRR comes from (a) StructTech's own engagements and (b) licensing OS modules to clients.

Why this phase exists (CRM Depth re-founded 7/14): What shipped for crm is a thin **sales-pipeline view**, not a CRM. Isaac is currently on the BMR app (which he's outgrowing) and StructTech OS's CRM must not be a downgrade. Full requirements grounded in docs/reference/CRM_DEPTH_REQUIREMENTS.md and LEAD_CONTROL_CENTER_SPEC.md — adopt the BMR spec's *features & logic*, NOT its UI/UX/workflow. Every entity gets full CRUD (SCOPE §2.6).

CORE PRINCIPLES (NON-NEGOTIABLE)

Source: SCOPE.md §2 — locked 7/8/26. These override every decision.

- **1. One build, licensed per tenant.** Never fork per client. Differences are config (entitlements + flags), not code.
- **2. Two axes of access:** *tenant entitlements* (which modules exist) × *user roles* (which screens a person sees). Nav bends to both.
- **3. No re-entry.** Each step is generated from the last — lead → estimate → signed → work order → materials/schedule → field → testimonial. The connective tissue is the product.
- **4. Field-first where it's a field tool.** Estimating and Field modules pass the "Roof Test": one-handed, gloves, bright sun, large touch targets (≥56dp), outdoor high-contrast mode (toggle, NOT default).
- **5. StructTech operates on top of client tenants** (agency layer) — it is not a peer tenant sitting beside them. Model as `agency_admin` membership in client org.
- **6. User-managed data — full CRUD, always (non-negotiable).** Every entity a user creates is theirs to **edit, delete, archive, void, or reassign from the UI**. No create-only happy paths. (Prefer soft-delete/archive where history matters.)
- **7. Configurable platform, tenant-authored (the core value prop).** StructTech OS is a **configurable engine, not a hardcoded app**. A tenant can self-onboard and shape the platform to *their* operation: their pipelines (multiple, of different types), their checklists/stages/fields, their pricing logic, their document templates. **Definitions live in per-tenant config the engine reads generically**; shipped sets (e.g. BMR's) are seeded defaults, not code. The data model must never hard-wire a single tenant's shape.

DESIGN SO AS NOT TO PRECLUDE (NORTH STAR — SCOPE §12)

Do **not** build these now, but do **not** make choices that block them:

- Business logic lives in server actions / RPCs, never buried in components — this action layer becomes the AI assistant's tool surface later.
- Append-only activity/history on every domain entity — it's the audit trail and the AI's "what changed" source.
- Estimate line items (Week 2) must allow an optional `product_id` link, not free-text only — client catalogs and the shop plug in here.
- Org isolation is the default, but don't hard-wire it so a controlled cross-tenant "published supplier catalog" (StructTech's live shop prices, readable by contractor tenants) becomes impossible.
- pgvector gets enabled when semantic search arrives — not now.
- Tenant-authored checklists/stages/fields + pricing matrix: per-tenant `tenant_modules.config` holds definitions; engine computes completion/derivation/gating generically. Stable scope keys for future pricing matrix.

TENANT MODEL, AGENCY LAYER & ROLES

Tenant Types (SCOPE §3)

internal (agency) — StructTech: Everything, plus the agency operator layer (switch into any client tenant) and StructTech-only modules (Scan/Roadmap, Delivery).

contractor — e.g. Brothers Metal Roofing: A licensed subset (Pipeline, Estimating, Coordination, Field) + its own StructTech-delivery view.

Agency Operator Layer (Wireframe 2a, 2h — confirmed 7/9)

Jacob logs in, lands in the StructTech workspace, and uses a **tenant switcher** ("■ Brothers Metal Roofing ■ back to StructTech", "viewing as: agency admin") to operate inside a client tenant. Login shows workspace selection when a user belongs to more than one.

Agency access = agency_admin membership model, NOT is_staff() (confirmed 7/9). StructTech operators see a client's data by holding an agency_admin membership row in that client org (my_org_ids() returns it) — no blanket bypass on new OS tables.

Multi-tenancy Model — the core of the whole build (CLAUDE.md)

- **Every domain table carries org_id.** RLS scopes every read/write by org via my_org_ids().
- **organizations** — one row per tenant. tenant_type ∈ (internal, contractor).
- **org_members** (existing — use as memberships) — (org_id, user_id, role). A user can belong to multiple orgs.
- **tenant_modules** (new) — (org_id, module_key, enabled, config jsonb). This is the entitlement layer. Nav and route guards render from it.
- **Two axes of access:** tenant entitlements × user role. Nav bends to both.
- **License activation = inserting rows** (organizations + tenant_modules + org_members), never provisioning a new database.
- **Escape hatch:** future enterprise client needing hard isolation can be siloed onto its own Supabase project with the same code.

Roles (per tenant) — SCOPE §5 (locked 7/9 — CHECK constraint; existing member kept valid)

Role	Description
owner	full tenant, dashboard, financials
admin	delegated admin (settings, users) short of owner
office	office/coordinator tier: pipeline, coordination, estimating
field	field/crew tier: Field module only; no pipeline, no dollar values (wireframe 2f)
client_portal_viewer	read-only delivery view + confirm own items; finalized when portal built
agency_admin	internal only — operate across client tenants (membership in client org)
member	legacy value, preserved for existing rows/features

MODULE REGISTRY & CLIENT JOURNEY

Module Registry (SCOPE §4 — the app suite)

Module	Who's Entitled	What it is	Wireframe
crm / Pipeline	all	Kanban with per-tenant stage config, deal panel, next-action + auto follow-up	1b, 2b
estimating	contractor	Live on-site estimate: preliminary (from lead) → on-roof validate → line-item present → sign	1c, 2c
proposals	internal only	Distinct from contractor estimating. StructTech's own quote/proposal builder + Present Mode for video sales calls (service tiers, engagement pricing, roadmap/investment slide). Not yet built.	—
coordination	contractor	Homeowner sign-off gate → work order → material list → schedule (material ready-by gates scheduling)	1d, 2d
field	contractor	Crew mobile: Today home, daily check-in, visual production packet (role = crew). Outdoor high-contrast default for this role. No pipeline, no \$ values.	2f, 3a
delivery	internal (operator) + surfaced read-only in client tenant	Engagement execution: levels/milestones/check-ins that drive the client portal	2i (admin) / 2e (client)
scan + roadmap	internal only	Public Revenue-Leak scan → auto-generated roadmap (already live)	—

The Client Journey (No Re-Entry) — Wireframe 2g (SCOPE §6)

Lead —generates→ **Preliminary estimate** —validated on-roof→ **Signed estimate**
—generates→ **Work order** —gates→ **Materials + schedule** —→ **Field / done** —→ **Testimonial**

Each artifact is generated from the prior one, never re-keyed. This flow is the spine of "absolute clarity across the client journey."

Module Entitlements by Tenant Type

internal (StructTech): All modules + agency operator layer + Scan/Roadmap + Delivery (internal admin view that drives client portal).

contractor (BMR example): crm (Pipeline), estimating, coordination, field + read-only StructTech Roadmap (Delivery) view inside their own tenant workspace. No Scan/Roadmap authoring, no proposals module.

APP SHELL, NAVIGATION & TENANT SWITCHER

Locked visual source: Wireframes 1a, 2a + Hi-Fi standalone.html. Match exactly. Same shell chrome, nav items driven by module entitlements per tenant. Accent = current section only.

StructTech Tenant (Full Entitlements) — Wireframe 1a

Top bar: Logo | ■ StructTech (agency view) | Search... | + Add | ■ | Avatar

Sidebar: Dashboard (active) | Pipeline | Scan / Roadmap | Delivery | Settings (bottom)

Main: Dashboard with KPI cards, portfolio view across all client engagements.

Brothers Metal Roofing Tenant (Limited Entitlements) — Wireframe 1a / 2a

Top bar (when switched): Logo | ■ Brothers Metal Roofing ■ back to StructTech | viewing as: agency admin | Search... | + Add | ■ | Avatar

Sidebar: Pipeline (active) | Estimating | Coordination | Field | StructTech Roadmap | Settings (bottom)

Main: Pipeline board (per-tenant stages).

Key behavior: The top-bar tenant switcher changes the active org context. `my_org_ids()` returns the switched org for RLS. Jacob operates inside BMR as `agency_admin` membership — no special-case bypass.

Login / Tenant Select — Wireframe 2h

Sign in screen → "You belong to 2 workspaces" → Select: StructTech (agency) or Brothers Metal Roofing.

After auth: workspace selection when user belongs to >1 org. Then lands in chosen workspace with correct nav.

Design System Notes (Hi-Fi + CLAUDE.md)

- **Accessibility:** small white-text buttons/chips use `--accent-strong`, not `--accent` (AA contrast). `--accent` is fine for large text and fills.
- **Field & estimating (mobile):** outdoor high-contrast mode (black bg) is **available via a toggle, NOT default-on** (updated 7/13 per Jacob — crews switch it on if they need it; default to the normal light theme); **≥56dp touch targets**; single thumb column; never show pipeline or dollar figures in the field role.
- **Sync status** is a first-class UI element on all field screens (offline / syncing / synced) — offline-first is a core principle.
- **Responsive:** WorkspaceShell desktop-first; mobile drawer nav + top bar needed (deferred, in progress 7/13).

PIPELINE / CRM & LEAD CONTROL CENTER

Wireframes 1b, 2b + CRM Depth phase (CLAUDE.md — re-founded 7/14). What shipped is a thin sales-pipeline view, NOT a CRM. Isaac needs the full Lead Control Center to run his whole sales process here instead of the old BMR app.

Pipeline v2 (per-tenant stages + next-action + follow-up) — Wireframe 2b

BMR funnel (config'd by tenant): Estimate Sent (with ■ follow up chip) | Negotiation | ...

Detail panel: Riverside Corp Campus | Stage: Estimate Sent | Value: \$18,400 | Next action: Call to confirm | Auto follow-up scheduled: Jul 10 (day 5) | Notes | Activity log | Quick actions: Call | Move stage

StructTech's own funnel is a different config, same board component: new_scan | contacted | call_booked | ... | closed_won

CRM Depth — Lead Control Center (the real build, staged)

Stage 1 (DONE — contact & address data): deals got lead_type / project_address / billing_address; new-lead form; estimate carries the address.

Stage 2 (Full lead data model — migration): first_name/last_name (split), secondary_phone, homeowner_or_contractor, remodel_or_new_construction, existing_roof_type, roof_type_requested, structured service address, referral_name, intake_checklist JSONB (main_issue, general_notes, site_visit scope data, estimate_inputs), milestone timestamps (site_survey_complete_at etc.), owner_id, tags. + RPCs.

Stage 3 (Command-stage + checklist engine — CRITICAL): Job-path stages (New Lead → Site Visit → Scope → Quote → Negotiating → Closed). Per-stage **vital fields**, the **two checklists** (intake-call + site-visit scope items), completion/progress (X/N, %), gating, recommended next action. "Form == checklist."

CRITICAL (SCOPE §12F): the stage/field/checklist DEFINITIONS live in per-tenant tenant_modules.config (same pattern as pipeline stages), seeded with BMR's set as the default — the TS engine READS config and computes completion/derivation/gating generically. NOT hardcoded per-tenant. Scope data points keep stable keys (for the future pricing matrix). Tenant-authoring UI + pricing matrix = backlog.

Stage 4 (Lead Control Center UI — 3-panel desktop/tablet + mobile): Replaces the thin deal panel. **Left:** quick actions + prospect data + owner + tags + pipeline-stage + revision history. **Center:** progress + command tabs + the active stage's form==checklist inline-edit rows. **Right:** author-stamped notes. Mobile stacks. Log Activity merges notes+activity. Full CRUD (§2.6).

Stage 5 (Ownership & attribution + BMR USER ACCOUNTS + SECURITY — go-live gate): Seed Isaac as owner (+ crew) so he logs in as himself, not Jacob's agency_admin. Tighten audit_leads policy, migrate legacy tables off is_staff() → my_org_ids(), RLS on structtech_state. Owner assignment dropdown, created_by on every note/edit, edit-by-ownership; salesman/manager roles.

Stage 6 (Scheduling): appointments (site_survey/inspection; scheduled_at/duration/status). Schedule quick action → completes the Site Visit stage.

Stage 7 (Scope → estimate wiring): The site-visit scope checklist feeds the estimate/quote (closes the site-survey→estimate loop).

Stage 8 (Views): table + calendar (kanban exists). Quick actions are native links now (tel:/sms:/mailto:); Twilio/Gmail later.

Done = clicking a BMR lead opens a working 3-panel command center: per-stage form==checklist with live completion, owner + author-stamped notes/activity, quick actions, schedulable site visit, the scope checklist flowing into the estimate — all editable/deletable (§2.6), fresh UI, matching Jacob's hi-fi. Isaac can run his whole sales process here instead of the old BMR app.

LIVE ESTIMATING & FIELD MODULE

Wireframes 1c, 2c (estimating) + 3a, 2f (field). Field-first: one-handed, gloves, bright sun, ≥56dp targets, outdoor high-contrast toggle (NOT default). No pipeline or \$ values in field role.

Live Estimating — Mobile Field Flow (Wireframe 1c / 2c)

- 1. Preliminary (auto-generated from lead):** ■ from lead: Riverside Corp Campus | 123 Elm St · Roof replace | [image placeholder] | \$14,200 – \$17,800 | est. from sq footage + roof type | "Start on-site visit" (56dp+)
- 2. On-roof validate & adjust:** Validate measurements | [photo/sketch area] | Roof squares 22.4 | Pitch 6/12 | Add: skylight flash +\$400 | "Recalculate"
- 3. Present number:** Final estimate | **\$16,650** | [line items breakdown] | "Present to client"
- 4. Sign (mobile signing state):** Sign to accept — \$16,650 | [signature capture area] | customer signs here | "Confirm & send copy"

Outdoor high-contrast toggle: Tap sun/moon icon (top-right). Normal (light) vs ■ Outdoor mode (black bg, white text, high contrast). Toggle available; default = normal light theme per 7/13 update.

Field Module — Crew Mobile (Role = crew, Wireframe 3a / 2f)

Black bg = outdoor high-contrast default for this role. Buttons ≥56dp, single thumb column, **no Pipeline, no dollar figures anywhere** in this role. Sync status first-class (offline/syncing/synced).

- 1. Today — assigned jobs list:** Today · Crew A | Tue Jul 9 — outdoor mode on | Riverside Corp Campus (Roof replace · Day 2 of 3) [Open job] | Elm St Residence (Gutter repair · not started) | ■ Daily check-in (big 56dp+ button)
- 2. Daily check-in — under 60 sec:** Check-in · Riverside | [photo capture tap area] ■ tap to add job photos | Hours today 7.5 | Materials used 18 panels ■ | Blockers? None ■ | "Submit check-in" (big button) | Sync indicator
- 3. Visual production packet:** Riverside — packet | ■ built from work order + sign-off photos | Trim map [annotated image with pins] | Boot / vent placements [image] | Custom detail callouts: ① Skylight flash — copper | ② Chimney boot — offset 6in

From phone testing (7/13–7/16): Estimate wizard back navigation (real bug — missing; steps must stay editable until present_estimate locks them). Coordination detail mobile layout cramped (stack/wrap needed). Cross-job crew schedule / Gantt (VITAL operational need — master timeline across ALL jobs/crews, not per-work-order).

Management Controls Retrofit (SCOPE §2.6 — full user CRUD, before Isaac goes live)

Earlier modules were built create-and-advance and lack user-facing edit/delete/archive on some entities. Backfill so the *user* manages their own data. High priority — needed before Isaac operates real jobs.

- **Deals** — user delete/archive + edit deal details (today: create + stage + notes only).
- **Estimates** — user void/delete + edit (the void status exists but has no UI path).
- **Work orders** — user delete/void a work order (materials + schedule already have delete; the work order itself does not).

Standing rule going forward: every new entity ships with edit + delete/archive from the start (Definition-of-Done item).

PROJECT COORDINATION & WORK ORDERS

Wireframes 1d, 2d. Signed job → homeowner sign-off (colors/finishes) → work order → materials (ready-by gates) → schedule (earliest start clamped to latest ready-by).

Coordination Flow (Wireframe 2d)

Signed job → Sign-off: colors & finishes → Work order → Materials → Schedule

Work order (from signed job) — Wireframe 1d: WO-1042 · Riverside Corp Campus | Signed Jul 5 · Roof replace, 22.4 sq | Material list: — Metal panels ×24 · ready Jul 12 | — Ridge cap ×6 · ready Jul 10 | — Underlayment ×3 rolls · ready Jul 9

Week-view scheduling board: Crew | Mon | Tue | Wed | Thu | Fri | Crew A [block on Mon–Wed] | Crew B [block on Tue–Wed] | hatched = blocked on underlayment (ready Jul 14) | red = blocked on materials

Material list — ready-by gates schedule (Wireframe 2d): — Metal panels ×24 · ready Jul 12 | — Ridge cap ×6 · ready Jul 10 | — Underlayment ■ delayed → Jul 14

Schedule board — earliest start clamped to latest ready-by: Visual Gantt-style with dependency enforcement. Editing start/end as two separate field-blur submits can transiently violate end_date >= start_date check (rare Postgres error in UI banner — fix in backlog: friendlier error or single two-field save).

From Deferred Items (BACKLOG)

- **Coordination schedule-block edit polish** — editing start/end date as two separate field-blur submits can transiently violate the end_date >= start_date check and surface a raw Postgres error in the UI banner. Rare, non-corrupting (final edit saves fine). Fix = friendlier error text, or a single two-field "save" instead of per-field auto-submit.
- **Cross-job crew schedule / Gantt (VITAL — operational, not visual polish)** — a master, company-wide timeline across ALL jobs and crews: see when a job ends so the right crew is ready for the next, spot gaps/overlaps, and see at a glance which crew is on which job now and where they'll be later. The current **per-work-order** crew+dates list does NOT give this cross-job resource view — this is a distinct master-schedule view/dashboard. Deferrable short-term (small crew) but a real need, priority for the scheduling-depth pass. (Stack: DHTMLX Gantt PRO decided for enterprise; portal has a deferred lightweight SVG Gantt.)

Empty State (Wireframe 1d / 4)

"No work orders yet — Work orders appear once a deal is signed and homeowner sign-off is complete."

CLIENT PORTAL, DELIVERY & ADMIN

Wireframes 1e, 2e (client), 2i (StructTech internal Delivery admin). Portal lives inside the client's own tenant — a nav item in BMR's workspace ("StructTech Roadmap" / Delivery), not a separate login. One workspace, one build. (Confirmed 7/8.)

Client Portal v2 — Expandable Milestones + Branding (Wireframe 2e / 1e)

Brothers Metal Roofing — engagement (white-labeled portal)

WIN CONDITION — Level 3: Structural sign-off by Jul 18

Level 1 [progress bar] ■ ASSESS — site survey (owner: PM)

Level 2 [progress bar] ■ BUILD — structural calc (owner: Engineer)

Level 3 ■ ● [active progress bar] ■ ADOPT — sign-off review (owner: Client) [client can attest ✓]

Level 4 ■ PROVE / HANDOFF

Next check-in: Jul 15 | Confirm / reschedule

Portal interaction (confirmed 7/8): read-only for level/milestone **status** (StructTech marks those), but the client **can confirm/reschedule check-ins and attest their own ADOPT/PROVE behaviors**. Amends the "read-only v1" line.

StructTech "Delivery" Admin — Internal Operator View (Wireframe 2i)

Internal view — drives what clients see on the portal.

Engagements in delivery: Brothers Metal Roofing — Level 3 (on track) | Riverside Corp — Level 1 (at risk)

■ Selected: Brothers Metal Roofing | [detail] | Mark milestone complete → updates client portal

Delivery Schema (already live — reuse)

engagements / engagement_levels / engagement_milestones / engagement_checkins (already seeded in structtech project). FK to organizations. StructTech marks status; client attests own behaviors.

DATA MODEL DIRECTION, RLS & SECURITY WORKSTREAM

Extend the existing structtech Supabase project — do not start a new database. Reality note (confirmed 7/9): organizations and org_members already exist and ARE the tenant/workspace concept. Extend them, never create a parallel org concept.

Foundation Tables — EXTEND, don't create (CLAUDE.md + SCOPE §9)

- organizations — add tenant_type (+ backfill). Already serves org_systems, tickets, org_invoices, org_invites, engagement_* FKs.
- org_members — expand roles (owner/agency_admin/office/crew). This **is** our memberships table. my_org_ids() reads from it.
- tenant_modules — (new) org_id, module_key, enabled, config jsonb. Per-tenant stage/checklist/field definitions live here.
- **Every domain table carries org_id** (backfill as separate migration).
- **Reuse (already live):** audit_leads, client_roadmaps, roadmap_playbook(), deals + follow_ups, engagements / levels / milestones / checkins.

Supabase + App Router — Mandatory Patterns (CLAUDE.md — every violation causes silent RLS failures, 404s, or auth bugs)

1. getSession() before every DB call. getUser() alone does not load the session; without it PostgREST sends no auth header and auth.uid() is null.
2. Always createClient() from @/lib/supabase/server in server contexts. Never import createServerClient directly.
3. **All inserts go through security-definer RPCs.** Direct .insert() on RLS tables fails even with valid session.
4. **All single-record fetches (by id) go through security-definer RPCs.** Direct .select().eq('id', x).single() fails RLS. Always pass org_id for RLS context.
5. List queries are fine with direct table access after getSession() — they resolve through the RLS helper.
6. Server actions redirect(), never return data.
7. All RLS policies use the existing my_org_ids() helper — never subquery org_members directly in a policy (infinite recursion).
8. Cannot find module './vendor-chunks/...' / ChunkLoadError = stale build cache: rm -rf .next && npm run dev.
9. R2 browser uploads require CORS on the bucket (GET PUT POST DELETE HEAD, AllowedHeaders: *, ExposeHeaders: ETag).

Security Workstream (REQUIRED before any client user gets a login — SCOPE §11)

- Tighten audit_leads "authenticated read = true" policy — currently any logged-in user can read every lead across all tenants. (The Week 2 org-scoped my_org_ids() read policy is a no-op until this is fixed.)
- Migrate legacy tables (org_systems, tickets, engagements, ...) off is_staff() to the my_org_ids() membership model.
- Enable RLS on structtech_state.
- Add NOT NULL to deals.org_id and the CRM/estimating org_id columns once RPCs are the only insert path.
- Replace tenant_type-based targeting with org_id in seeds/updates once a 2nd contractor org exists.

HARD prerequisite before client logins: tighten audit_leads, migrate legacy tables, RLS on structtech_state. Then seed Isaac as owner + crew.

BUILD PHASES, DEFINITION OF DONE & CURRENT PHASE

*Source: CLAUDE.md CURRENT PHASE + BUILD_PLAN_3WEEK.md (implied) + SCOPE §2.6 Definition of Done.
Weeks 1–3 COMPLETE and live at os.structek.com.*

Weeks 1–3 COMPLETE (Foundation + Pipeline + BMR Estimating + Coordination + Field + CRM Depth)

Week 1 — Foundation: auth/multi-tenancy/shell, extend organizations (tenant_type), extend org_members (roles), add tenant_modules, my_org_ids() RLS helper, org-scoped policies, security-definer RPCs. Backfill org_id onto existing tables (separate migration).

Week 2 — Pipeline both tenants + BMR live estimating: per-tenant stages, deal panel, next-action + auto follow-up, live on-site estimate flow (preliminary → validate → present → sign), line items, signatures, R2 prep.

Week 3 — BMR coordination + field + management-controls retrofit: work order → materials (ready-by gates) → schedule, crew mobile (Today/check-in/packet, outdoor mode, sync status), full user CRUD on Deals/Estimates/Work orders so Isaac can fix his own data without SQL.

CRM Depth (re-founded 7/14, running in parallel/staged): Lead Control Center (3-panel command center), full lead model + intake_checklist JSONB, command-stage + checklist engine (config-driven from tenant_modules.config), ownership/attribution, scheduling (appointments), scope → estimate wiring. **Done** = Isaac can run his whole sales process in the new UI instead of the old BMR app.

Definition of Done (every phase — non-negotiable)

- TypeScript builds clean; npm run dev runs without console errors.
- RLS verified: a user in org A cannot read org B's rows (write an explicit check).
- Matches the locked hi-fi visually.
- Behavior confirmed with the user before anything ambiguous ships.
- **Full user CRUD (SCOPE §2.6):** every entity the phase creates can be **edited and deleted/archived/voided by the user in the UI** — not just created and advanced. If the user could make it, the user can fix or remove it, without a developer or SQL. No create-only happy paths.

Rules of Engagement (CLAUDE.md — non-negotiable)

- **Build only inside structtech-os/.** Never edit anything outside this folder. The legacy projects (../structtech/, ../Brothers Metal Roofing/) are not accessible and are reference only.
- **Build the current phase only.** Right now that is Week 1–3 + CRM Depth staged work. Do not build Week 2+ features unless told.
- **Do not fork the BMR app or structtech-pipeline.** Reuse *patterns*, never copy a whole app. The duplication between those two is exactly what this build ends.
- **Confirm before behavior-changing decisions.** If the user describes a behavior, confirm it's intended before "fixing" it. Never guess at product behavior.
- **Ask before:** applying any migration to the live Supabase project, adding a new dependency, changing auth/RLS/security, or anything touching structtech_state.
- **Small, reviewable commits.** One concern per commit. The user reviews diffs.

BACKLOG, DEFERRED ITEMS & PHONE TESTING FIXES

Source: BACKLOG.md (durable queue — last updated 7/13/26 after Week 2). Treat as the source of truth for "what's owed and what's next." When an item is picked up, move it into the active CLAUDE.md phase; when done, delete it here.

Next up — Revenue Layer (owed from Week 2 "close-tool" fork)

proposals + Present Mode module (internal-only; SCOPE §4). StructTech's own quote/proposal builder + video-call **Present Mode**. The counterpart to contractor estimating — this is how StructTech closes its *own* paying clients (\$20K MRR path). Source material: docs/reference/EXISTING_CRM_SCHEMA.md (proposals/quotes) + the legacy PRESENT_MODE_PLAN.md. **Not started**. Was consciously deferred when BMR live estimating was prioritized in Week 2 — not dropped.

Deferred from Week 2 (small, tracked)

- **Per-deal next-action chip** — add next_action (text) + next_action_at (date) to deals, an editable field, and the card/panel chip (wireframe 1b/2b element).
- **R2 file storage** — persist estimate PDFs and populate signatures.pdf_url; today PDFs generate on-demand and pdf_url stays null.
- **Follow-ups have no home right now (dormant feature)** — auto day-2/day-5 follow_ups are still *scheduled* in the DB, but (a) sending was never built (the Make.com scenario is "later"), and (b) the Stage-4 Lead Control Center dropped the follow-up *display* the old DealPanel had. So they're created, invisible, and inert. Needs a home — a tasks/today view (see BMR's "Coach's Calls") and/or re-surfacing on the lead — plus the Make sender.
- **Responsive mobile web shell** — WorkspaceShell is desktop-only; needs drawer nav + mobile top bar. (In progress as of 7/13.)
- **Coordination schedule-block edit polish** — see Page 9.

From Phone Testing (7/13–7/16) — UX, Scheduling, Docs

- **Estimate wizard: back navigation** (fixing near-term — real bug) — the preliminary→validate→present→sign flow has no way back; missing a field before advancing forces starting a whole new estimate. Add back/step navigation (steps stay editable until present_estimate locks them).
- **Coordination detail: mobile layout** (fixing near-term) — material rows (name/qty/date/x) and schedule crew+date fields are cramped/jammed on a phone. Stack/wrap for narrow viewports.
- **Cross-job crew schedule / Gantt (VITAL — operational)** — master company-wide timeline across ALL jobs and crews (see Page 9).
- **Intermittent session bounce to /select-workspace (INVESTIGATE — real-user risk)** — recurs across sessions: a mid-interaction request occasionally redirects to the workspace picker (looks like a transient session/auth-refresh miss in middleware or getSession). Harmless in testing (re-navigate), but a user bounced mid-form loses their input. Not a one-off — worth a real root-cause pass (middleware refresh timing / cookie race).
- **Docs section** — one place to view / download / send work orders + estimates (signed or not), later invoices + packets. Ties to the tenant-customizable document-template system (SCOPE §12E). Jacob: "add to later build if needed."
- **From 7/16 Lead Control Center fix-pass:** Roof-type multi-select needs a touch-friendly picker (native <select multiple> requires ctrl/cmd-click; acceptable v1 per "editable later via the editor," but a proper chip/checkbox picker is the real fix). Two legacy BMR deals hold project_address only (no structured address) — read-only "(legacy)" hint in EditLeadDetailsForm so Isaac can re-enter structured; once done, free-text value is unused.

FUTURE NORTH STAR, INTEGRATIONS & NON-GOALS

Source: SCOPE §12 (Future / North Star — not v1, foundation must not preclude) + §13 (Integrations — committed, staged for future) + §10 (Non-goals). Do not build now, but every foundation decision must allow them to drop in without a rebuild.

A. AI assistant + semantic search

Conversational read/update of projects ("what's the status of X" / "update Y with...") behaving like a personal assistant.

Why the foundation already supports it: every mutation is a named security-definer RPC (that becomes the assistant's tool surface); org-scoped RLS means the assistant runs *as the user* and can't leak across tenants; append-only activity gives it a timeline to summarize; Supabase pgvector powers semantic search (enable when we get there).

Preserve now: RPC-as-action-layer and append-only activity on every entity.

B. Client product catalogs + inventory

Clients add their full product list for accurate pricing and material/inventory tracking. Just another org-scoped module (products/pricing/stock), entitled per tenant.

Preserve now: estimate line items (Week 2) must allow an **optional product_id** link, not free-text only — that nullable link is how catalogs plug into pricing later.

C. StructTech shop / distribution integration — the vertical-integration payoff

StructTech's online metal-roofing supply shop, baked in: a roofer's estimate prices against StructTech's **live shop prices**, and once the job is signed they place the order in one click. Prototype already exists in ../structtech-x-windyhill (Stripe, supplier checkout, catalog, configurator).

New concepts to model when built (don't preclude): Controlled cross-tenant catalog (StructTech's published prices readable by entitled contractor orgs — deliberate exception to strict org isolation); Price snapshot (estimate lines capture the price at time-of-estimate while referencing the live price for reorder); Order/commerce domain extends the no-re-entry journey.

D. Native mobile app (Apple App Store + Google Play)

The web app doesn't ship to app stores — a native app does. Stack already plans **React Native + Expo + PowerSync** for the field-first modules (estimating, field/crew): a genuinely native, offline-first client deployed to both stores via Expo EAS.

Why the foundation already supports it: everything is API-first (Supabase + security-definer RPCs + org-scoped RLS), so the native app is just another client on the same backend — no rewrite. Web for office/desktop, native for the field (camera, offline-on-a-roof, push — also what clears Apple review vs. a thin web wrapper).

Model: one multi-tenant StructTech OS app; users log into their workspace (not a per-client app — white-label branding lives inside the one app).

Phase: distinct workstream AFTER the web platform + go-live (see docs/GO_LIVE_PLAN.md §4).

E. Tenant-customizable document templates

Each tenant (and StructTech) customizes templates for **estimates, invoices, work orders, proposals, and contracts** — branding (logo, colors, company info), layout, and terms. Documents render from those templates to PDF (stored in R2).

Why the foundation supports it: documents render from structured data (estimates/line-items/signatures already model the data) through a swappable renderer.

Preserve now: keep PDF/document generation in an isolated, **tenant-parameterized renderer** (branding + terms pulled from config), never a hardcoded single-tenant layout — so the editable-template system drops in later without a rewrite. signatures.pdf_url (and future equivalents) stay nullable until R2 storage is set up.

F. Tenant-authored checklists/stages + the pricing matrix (the true multi-tenant payoff)

Every tenant runs their business their own way, so **each tenant authors their own command stages, checklist fields, and data points** — not CC hardcoding one roofer's set. And a **pricing-matrix / estimate-logic tool** lets a tenant map specific checklist data points (from any checklist) to pricing rules ("this data point costs \$X per unit"), so **the estimate is generated from their custom checklist data**.

Why the foundation supports it: checklist *values* already live in the flexible intake_checklist JSONB; per-tenant *definitions* live in tenant_modules.config (same pattern as pipeline stages); estimate line items already carry a nullable product_id to link a data point to a priced product.

Non-negotiable now: checklist/stage/field **definitions must be per-tenant CONFIG DATA the engine reads, NOT hardcoded** — the TS engine computes completion/derivation/gating generically over config; the BMR set is the *seeded default*, not code. Scope data points keep **stable keys** so the pricing matrix can reference them. The authoring UI + pricing-matrix tool are backlog; the data model must not preclude them.

Integrations (committed — staged for future build, SCOPE §13)

1. Google Workspace — Accounts + Calendar + Gmail. Two-way sync for dated entities; outbound for CRM follow-up sequences and automated client updates.

2. Twilio — SMS — Instant lead-response text-back, automated client update texts, crew schedule push.

3. Stripe — payments + commerce — StructTech's own MRR (tenant subscriptions, per-level engagement billing); shop checkout when a roofer orders materials after signature.

Runner-up queue: QuickBooks Online (accounting sync); Aerial roof measurement (EagleView / Hover — auto-generates preliminary estimate from address).

Non-Goals / Out of Scope (SCOPE §10)

- **Not** the enterprise Procore-competitor construction platform. The controller skill's deep construction modules (RFIs, submittals, AIA pay apps, P6 Gantt, BIM) are **future module candidates**, not v1.
- **Not** rebuilding "The Field" as a separate product — its concepts become the field and coordination modules.
- No AI layer, no per-level invoicing, no computer vision in this scope.
- The codebase anchor (BMR app as OS base) is **provisional** pending this week's BMR fixes; decided after those settle.

TECH STACK, PATTERNS & GLOSSARY

Tech Stack — Decided, Do Not Re-Litigate (CLAUDE.md + SCOPE §8)

Layer	Choice
Web	Next.js 14 (App Router) + TypeScript
Styling	Tailwind CSS (theme mapped to design tokens)
Backend	Supabase (Auth + Postgres + RLS + Realtime) — existing structtech project
Files/photos	Cloudflare R2 (S3-compatible)
Mobile (later)	React Native + Expo + PowerSync/SQLite
Email	Resend
PDFs	pdf-lib
Scheduled sends	Make.com scenario

All secrets via env vars (.env.local). Never hardcode keys or project refs in source.

Key Glossary (from source docs)

my_org_ids() — existing security-definer helper that returns the set of org_ids the current user belongs to (reads org_members). Every RLS policy uses it. Never subquery org_members directly in a policy.

tenant_modules.config — per-tenant JSONB where stage definitions, checklist definitions, field definitions, and (future) pricing rules live. The engine reads it generically; BMR's set is the seeded default.

intake_checklist — JSONB column on deals/leads holding structured qualification + site-survey data (main_issue, general_notes, scope data points with stable keys for future pricing matrix).

Lead Control Center — the 3-panel (desktop) / stacked (mobile) command center that replaces the thin deal panel. Every row is a form field AND a checklist item at once. Dual-track model, per-stage vital fields, two checklists, completion %, gating, recommended next action.

Agency admin — internal-only role: holds a membership row in a client org so my_org_ids() returns it and RLS works without special cases. Top-bar switcher changes active org context.

Full CRUD (SCOPE §2.6) — every entity a user can create, the user can also edit/delete/archive/void from the UI without developer/SQL. Definition of Done item for every phase.

Roof Test — field/estimating UI must work one-handed, gloves on, bright sun: ≥56dp touch targets, single thumb column, outdoor high-contrast mode available (toggle), no pipeline/\$ in field role.

No re-entry — each artifact (estimate, work order, etc.) is generated from the prior one; data is never re-keyed by the user.

END OF BLUEPRINT — This document is a faithful 15-page synthesis of CLAUDE.md, SCOPE.md (locked 7/8/26), BACKLOG.md (7/13/26), wireframes (1a–1e, 2a–2i, 3a), and the locked hi-fi standalone.html. All features, layout decisions, rules, data model direction, security patterns, and non-goals are preserved exactly as outlined. No information was changed, added, or omitted from the source directives.

Next actions live in the active CLAUDE.md phase section + BACKLOG.md durable queue. Build only inside structtech-os/. Confirm before behavior changes. Small reviewable commits.